

Young, Dynamic Julia for Numerical Computing

JAI SACHITH PAUL AND RESHMI DAS

In this article we introduce you to three-year-old Julia, a high-level, high-performing dynamic programming language. Julia can be used for scientific computing, for general-purpose programming or even as a specification language. Syntax of this language is quite similar to that of other popular technical computing environments. The language allows the user to automatically generate efficient and specialised codes for various argument types.

While the base library is written mostly in Julia itself, the language also makes use of powerful C and Fortran libraries for linear algebra, random number generation, signal processing and string processing. The latest stable release of Julia, version 0.3.11, licensed under MIT licence, is bundled in this month's EFY Plus DVD.

Motivation behind Julia

When it comes to scientific computing, users prefer a high-level dynamic programming language. Instead of spending time worrying about managing features like the memory, or loops, they can then use it more productively to play around with matrices, large data sets, user-friendly plots and the like.

Unfortunately, the design of most well-known dynamic programming languages is not suitable for high performance. If the user is unable to vectorise the code and have built-in functions operating on these, the code will be extremely slow and he or she will be in trouble.

In order to resolve this issue, the user has to switch over to a low-level

Julia at a glance

Key features: Multiple dispatching, high-speed dynamic programming, support for parallel and distributed computing, efficient Unicode support and much more

Latest stable release: Version 0.3.11

Licence type: MIT licence

Developers: Jeff Bezanson, Stefan Karpinski, Viral B. Shah and their team

Supporting OS: Linux, OS X, FreeBSD and Windows

For more information: juliadlang.org

language such as C or Fortran, write key computational kernels and call these from high-level languages. A code written in C or Fortran would neither run efficiently nor be glued easily in a high-level language.

A just-in-time compiler called LLVM allows the user to write the high-level code in Julia itself, thereby helping to achieve performance equivalent to C or Fortran.

Some notable features

Programmers can write clear, high-level, abstract codes like they do in high-level dynamic languages, without having the licensing headaches as in MATLAB.

Julia is written mostly in Julia. Therefore the user can look inside the code, see its implementation, as it is more readable and comprehensible than a high-level dynamic language implemented in C or Fortran.

Multiple dispatch. A method defines one possible behaviour of a function. Selection of the method for execution when a function is applied is called dispatch. In traditional object-oriented languages, dispatch is generally based on the first argument, whereas Julia makes use of multiple dispatch techniques, where all arguments of a function are used for the selection of the method.

When it comes to numerical com-

```
function mandel(z)
    c = z
    maxiter = 80
    for n = 1:maxiter
        if abs(z) > 2
            return n-1
        end
        z = z^2 + c
    end
    return maxiter
end

function randmatstat(t)
    n = 5
    v = zeros(t)
    w = zeros(t)
    for i = 1:t
        a = randn(n,n)
        b = randn(n,n)
        c = randn(n,n)
        d = randn(n,n)
        P = [a b c d]
        V[i] = trace((P.*P)^4)
        W[i] = trace((Q.*Q)^4)
    end
    std(v)/mean(v), std(w)/mean(w)
end
```

A sample Julia code

puting related to mathematical operators, this approach seems to be pretty useful. When we add two arguments, say, x and y (that is, x+y), we cannot say that the addition operation belongs more to x than to y.

Similarly, in the case of other numerical operations, multiple dispatch techniques seem to be more relevant than conventional dispatch techniques.

Parallel computing. Programs written in Julia can run on multiple processors in separate memory domains at once, with the help of a